Lab 5: Design sequential circuit

Assignments

Continuous assignment:

- Continuous assignments continuously monitor the RHS (right – hand side) and update the LHS (left – hand side) immediately when the RHS change, does not introduce any timing delays.
- Multiple continuous assignment executes concurrently, order of statements is not important.
- Used outside **always** statement to design **simple combinational circuit.**
- *Syntax: assign a = b;*

Blocking assignment:

- Cause the RHS expression to be evaluated immediately, and the LHS variable is updated before moving to the next statement.
- Multiple blocking assignments are executed sequentially in the order they appear in the code, order of statements is important.
- Used inside ***always*** statement to design **combinational circuit.**
- *Syntax: a = b;*

Non – blocking assignment:

- Updating for the variables on the LHS occur at the end of current time step, typically on a clock edge.
- Multiple non-blocking assignments executes concurrently, order of statements is not important.
- Used inside ***always*** statement to design more **complex sequential circuit.**
- *Syntax: a <= b;*

Compare between types of assignments

	Continous assignment	**Blocking assignment**	**Non-blocking assignment**
Update LHS to RHS	Immediately	Immediately	At the end of current time step, typically on a clock edge
Order of execution statements	Concurrently	In order	Concurrently
Using to design	Simple combinational circuit	Combinational circuit	Sequential circuit
Inside or outside ***always***	Outside	Inside	Inside
Syntax	assign a = b	a = b	a <= b

Always statement

Syntax:

always @(sensitivity list)
statement;

- Statements in *always* statement is executed only when the event specified in sensitivity list occurs.
- *Always* statement can be used to design flip-flop, latches or combinational circuit depend on the sensitivity list and statement.

- Types of always statement:

+ **always @(sensitivity list)** can be used to design sequential or combinational circuit based on sensitivity list

- **always_latch** is dedicated for design latch
- **always_ff** is dedicated for design flip flop

To design D flip-flop: always@(posedge clk)

Statement

	always_ff (Flip-flop)	**always_latch (Latch)**
Update	Đồng bộ theo cạnh clock	Không đồng bộ, hoạt động theo tín hiệu enable
Syntax	always_ff @(posedge clk) hoặc @(negedge clk)	always_latch
Using to design	Dùng cho các mạch đồng bộ	Dùng cho các mạch không đồng bộ
State change	Thay đổi tại cạnh xung clock	Thay đổi bất kỳ khi nào có enable
Applications	Thanh ghi, bộ đếm, mạch trạng thái	Mạch giữ dữ liệu, các mạch điều khiển giữ trạng thái

Common used statement

for	while
for (initialization; condition; increment) begin // Loop body end	while (condition) begin // Statements to execute while the condition is true end
if – else	**case**

if (condition) begin
  // Statements to execute when the condition is true
end
else begin
  // Statements to execute when the condition is false
end
	case (expression)
	 value1: begin
	  // Code for when expression matches value1
	 end
	 value2: begin
	  // Code for when expression matches value2
	 end
	 // More value cases...
	 default: begin
	  // Code for when no value case matches the expression
	 end
	endcase

D flipflop

Design D flip flop using always_ff and non-blocking assignment.

D flipflop truth table

clk	d	q
↑	1	1
↑	0	0
other values	x	previous value

Ex:
```
always_ff @(posedge clock) begin
    // Your sequential logic here
    end
```
- Using non-blocking assignment.

D- flipflop

d_flipflop.sv

```
module d_flipflop(input logic clk, d,
                  output logic q);
 always_ff@(posedge clk)
  begin
       q <= d;
  end


endmodule
```

d_flipflop_tb.sv

```
module d_flipflop_tb();

                logic clk, d, q;
                d_flipflop dut(clk, d, q);

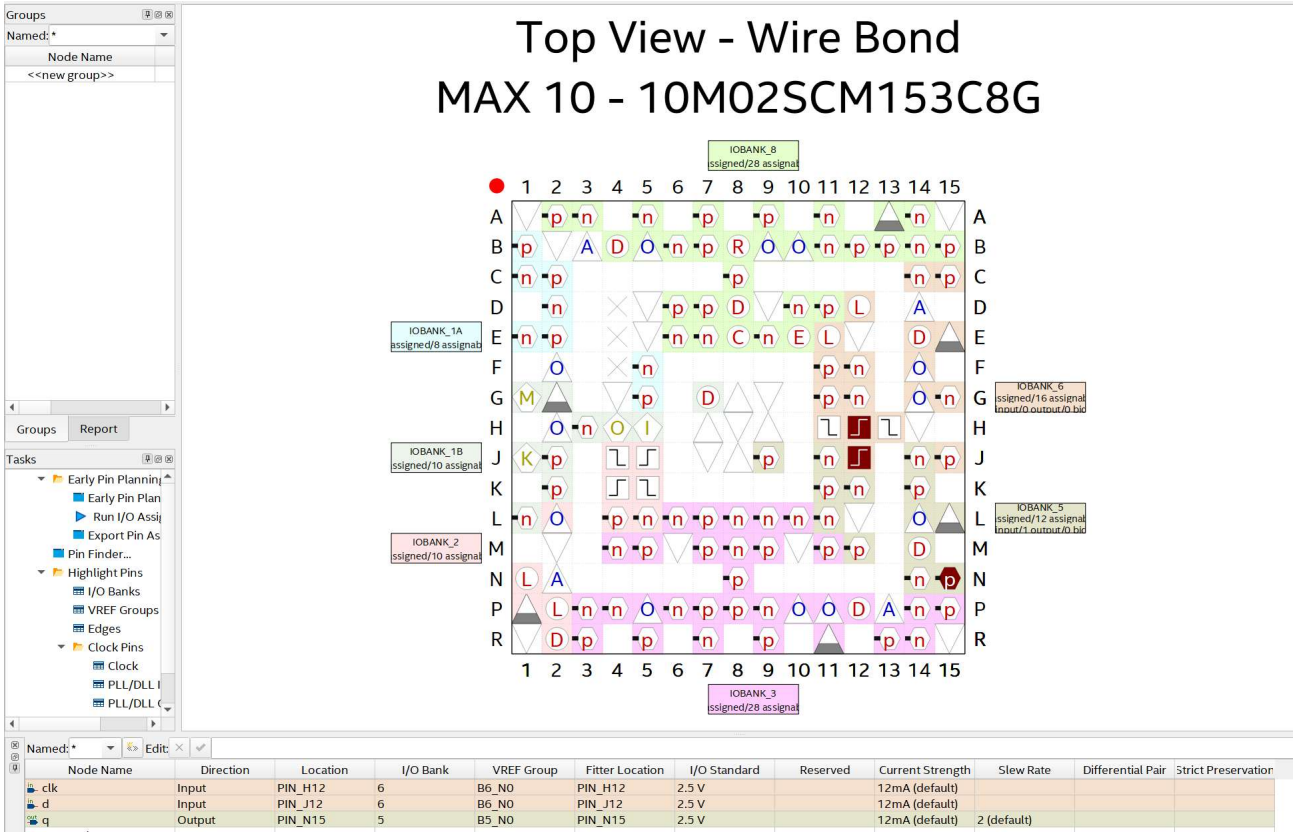
                //Clock signal generation
                always begin
                                clk = ~ clk; #5;
                end

                initial begin
                                clk = 0;
                                d = 0;

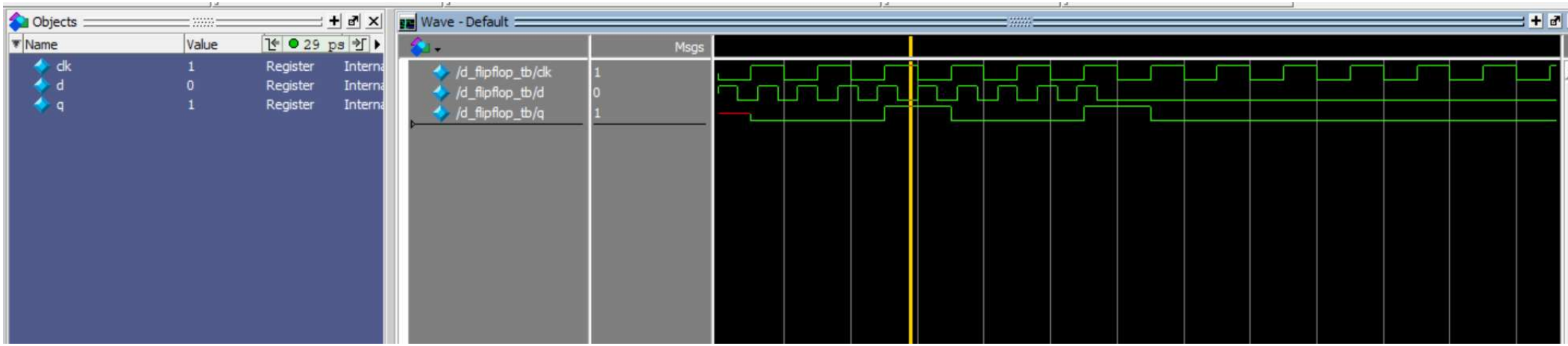
                                for (int i = 0; i < 20; i++)
                                begin
                                                d = ~ d; #3;
                                end
                                $stop;
                end
                endmodule
```

Test D-flipflop on STEP-MAX10 board

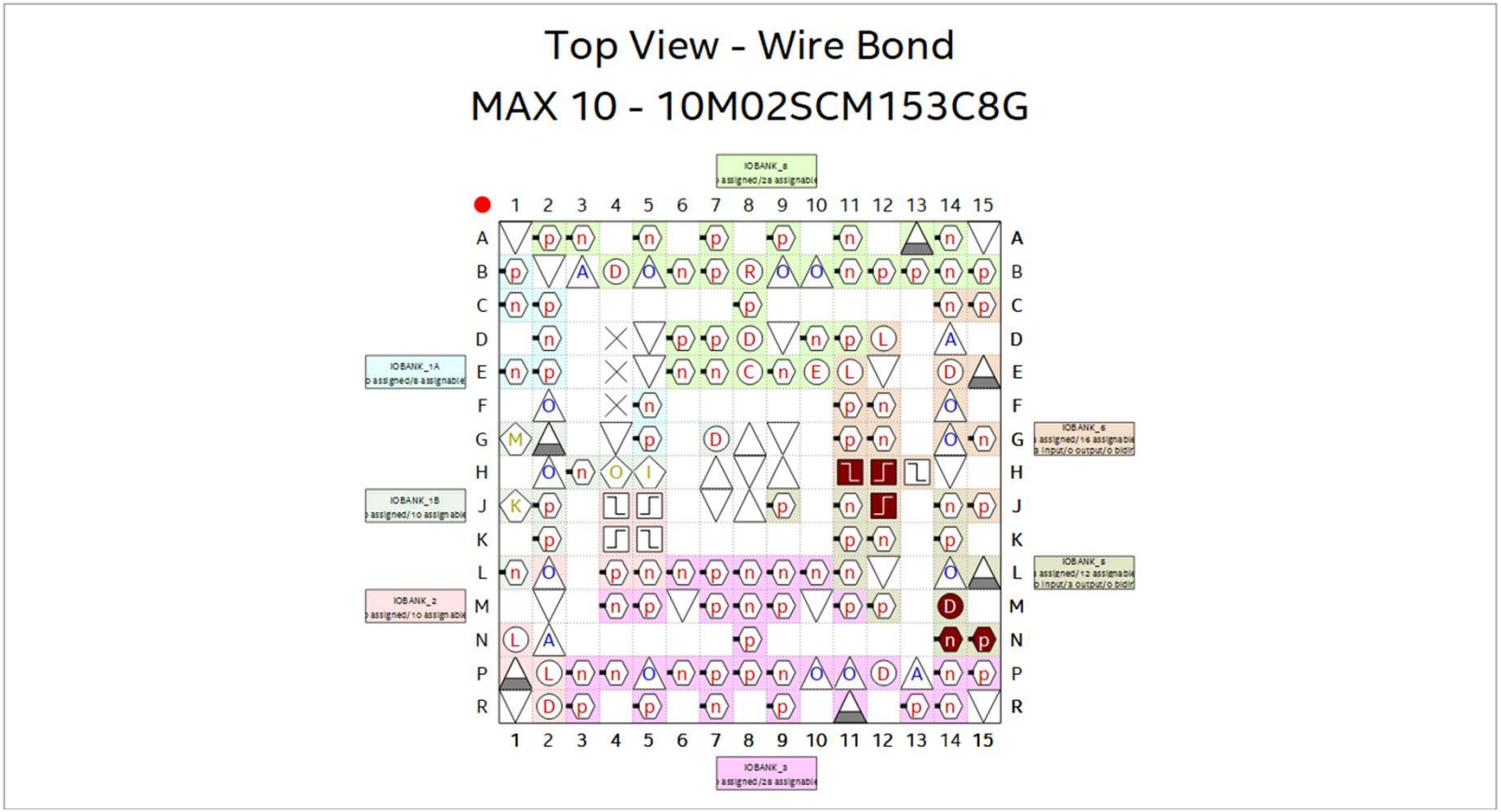
Method 1: Assign clock signal to a switch on STEP – MAX10 board. Using switch to create clock signal.



D- flipflop simulation result

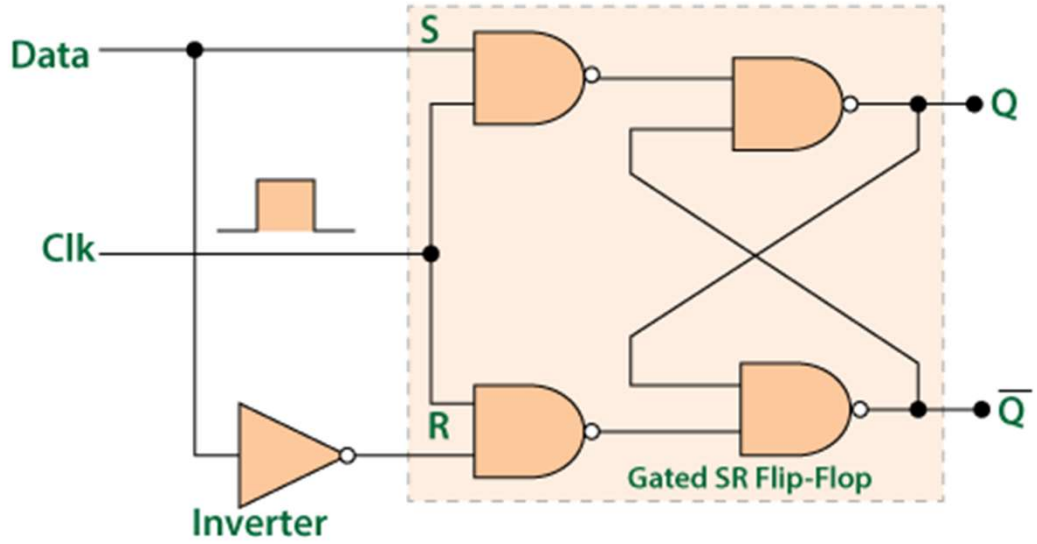


Pin assignments



D latch

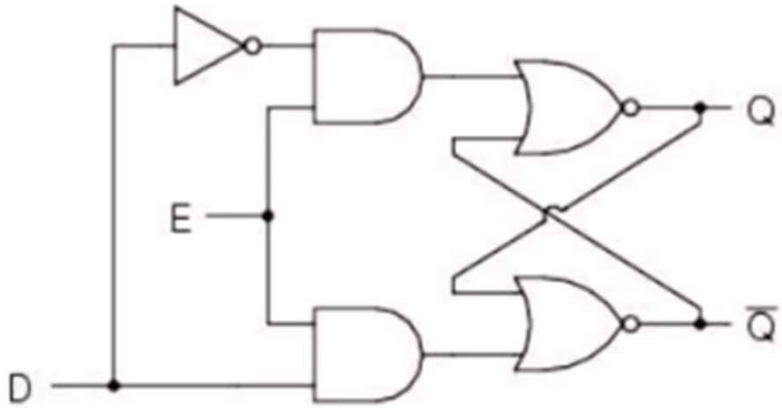
Ex1: Design an D latch having following truth table using in SystemVerilog and simulate on ModelSim.



Enable	D	Q_n	Q_{n+1}	STATE
1	0	x	0	RESET
1	1	x	1	SET
0	x	x	Q(n)	No Change

D flipflop

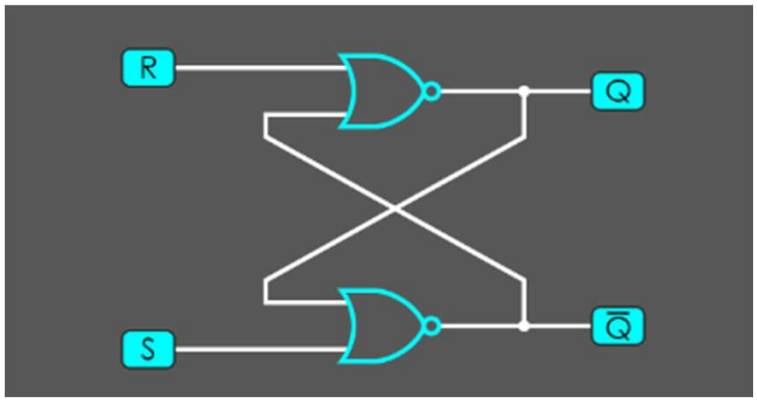
Ex5: Design an D flipflop having following truth table using in SystemVerilog and simulate on ModelSim.



Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

S-R latch

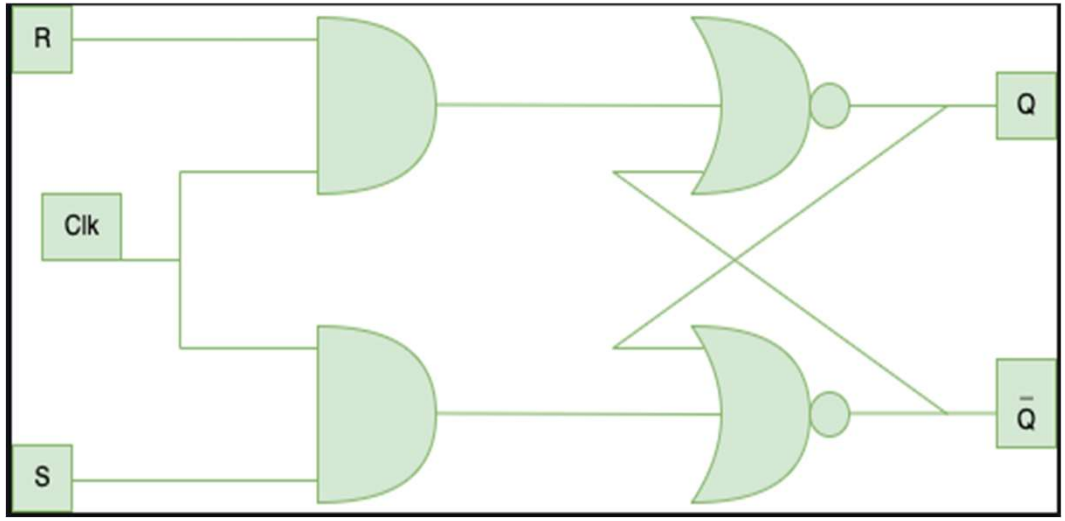
Ex2: Design an S-R latch having following truth table using in SystemVerilog and simulate on ModelSim.



Enable	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	X

S-R flipflop

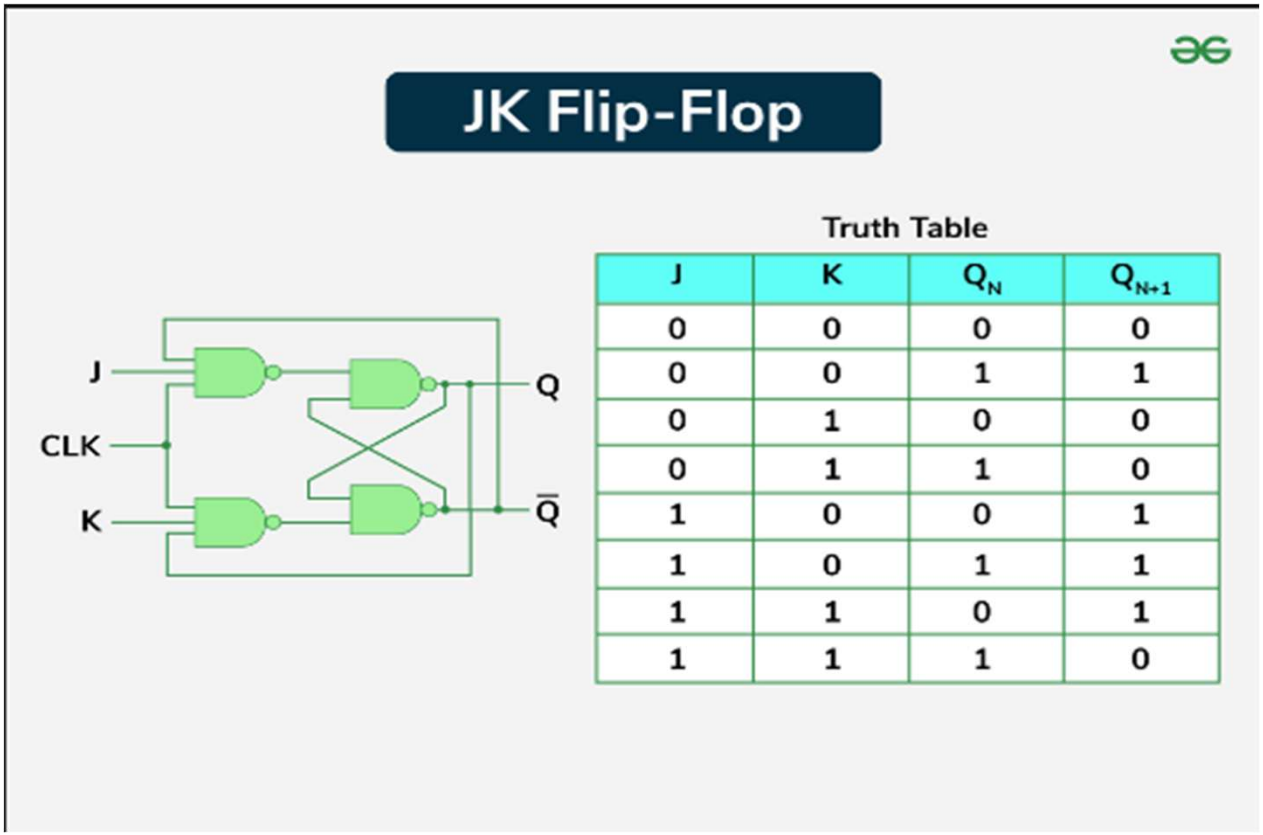
Ex4: Design an S-R flipflop having following truth table using in SystemVerilog and simulate on ModelSim.



S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

JK flipflop

Ex3: Design an JK flipflop having following truth table using in SystemVerilog and simulate on ModelSim.



J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0